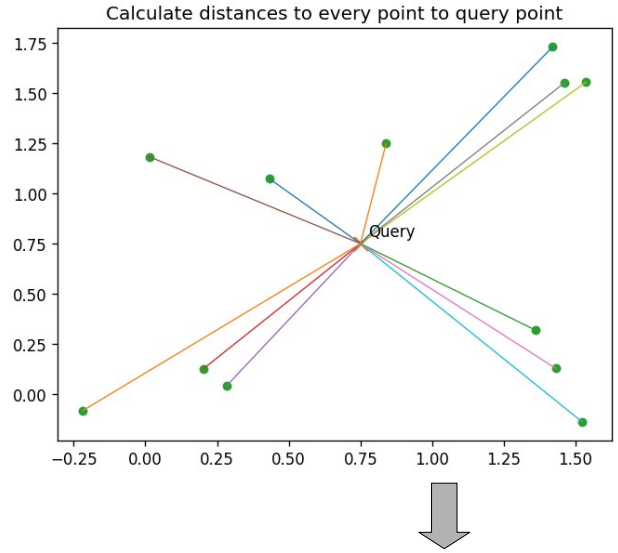
Similarity Search in Vector Databases: kNN, IVF, NSW, HNSW

An overview

K Nearest Neighbors (kNN)

- Finds the k closest points to a query
- By computing distances to all n points
- O(n) per query → slow at scale
- Will find best matches

kNN- algorithm



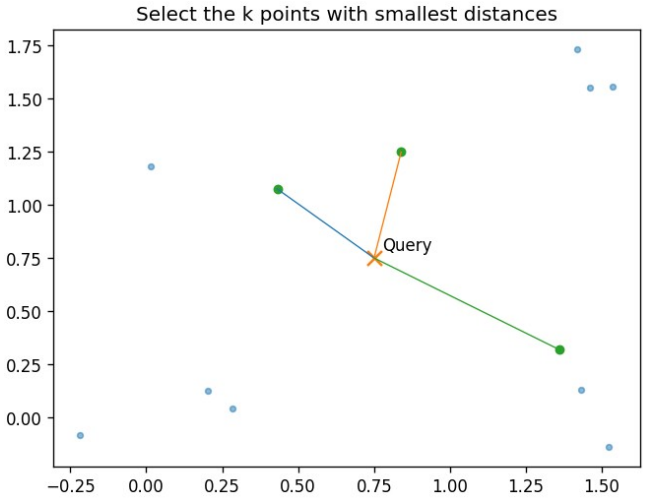
Distance
1.275413
0.847395
0.829449
0.746751
0.921099
1.174519
1.072903
1.187165
1.125681
0.454620
0.853182
0.505817

Distances

Distance
0.454620
0.505817
0.746751
0.829449
0.847395
0.853182
0.921099
1.072903
1.125681
1.174519
1.187165
1.275413

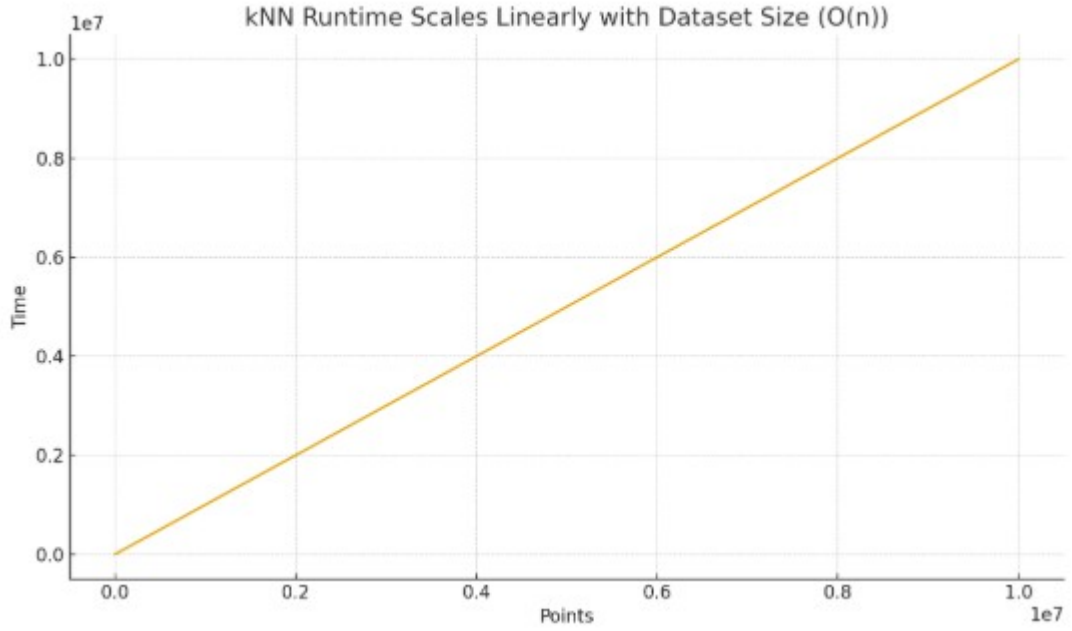
Sorted Distances

Select k closest



Why kNN Doesn’t Scale

- Per-query time grows linearly with data size: kNN must compare a query to every point → O(n) work per query
- latency explodes as n grows.

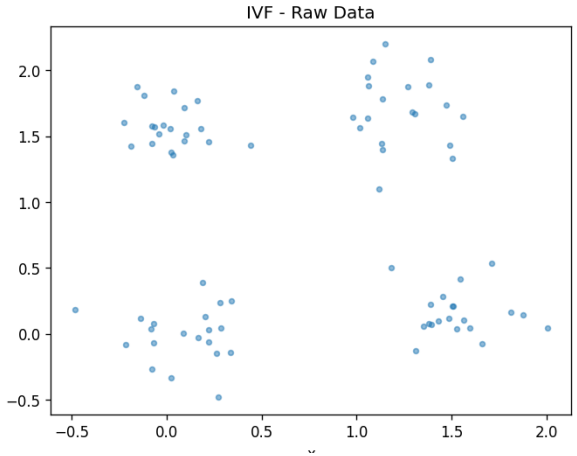


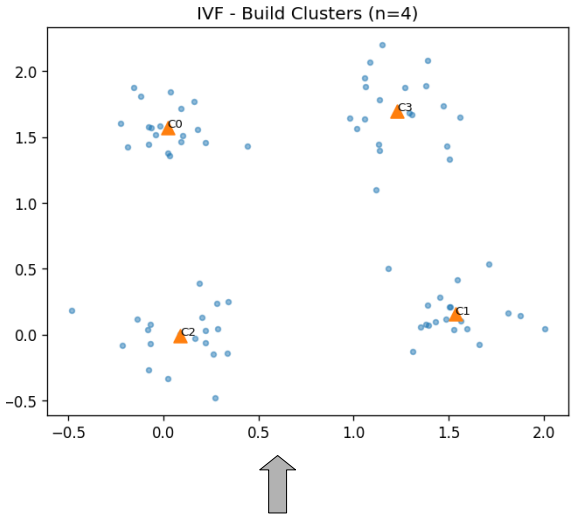
IVF: Inverted File Index — Idea

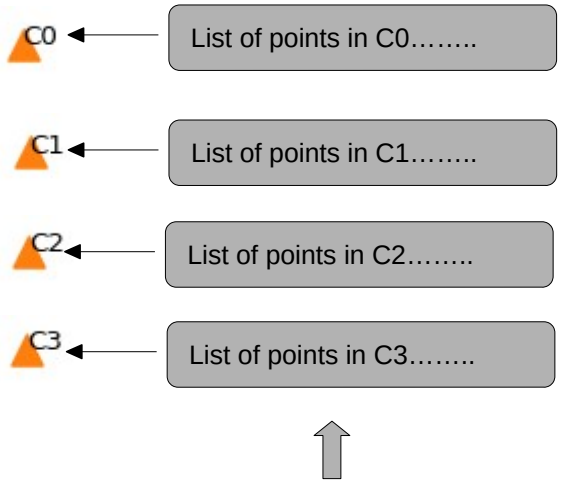
- Partition space into n clusters (n_probe)
  - Rule of thumb: 1000 clusters for 10000000 points -
- At query time probe only k closest clusters. Pick closest point(s) in each.
- So if k=2 and have 1000 clusters, then just search 2 out of 1000 for relevant points

IVF: Build Clusters

Partition space into n_probe=4 clusters, build list of points in each cluster (n_clusters is a hyperparameter)







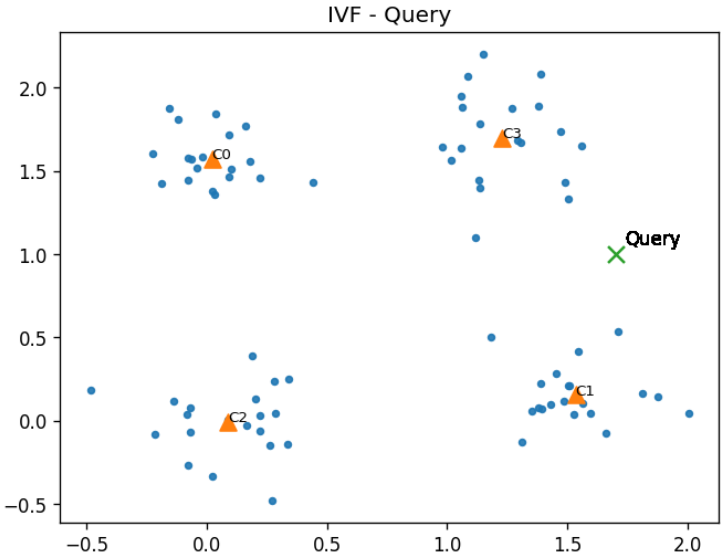
This is an iterative process (KMeans)

1) Randomly place 4 centroids
2) Assign each point to its closest centroid
3) Move centroid to center of its points
4) Go to 2 until no points change cluster

Have list of all points in each cluster

IVF: Query time

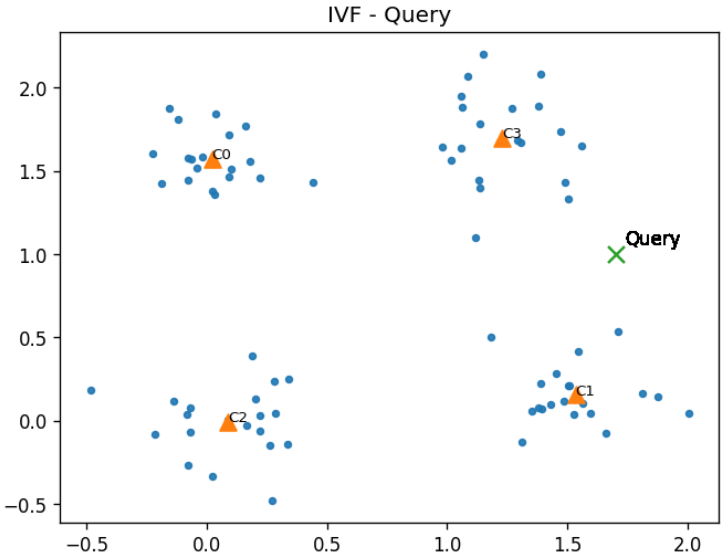
Ex. Find closest 3 points to Query



IVF: Query time

Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?



IVF: Query time

Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

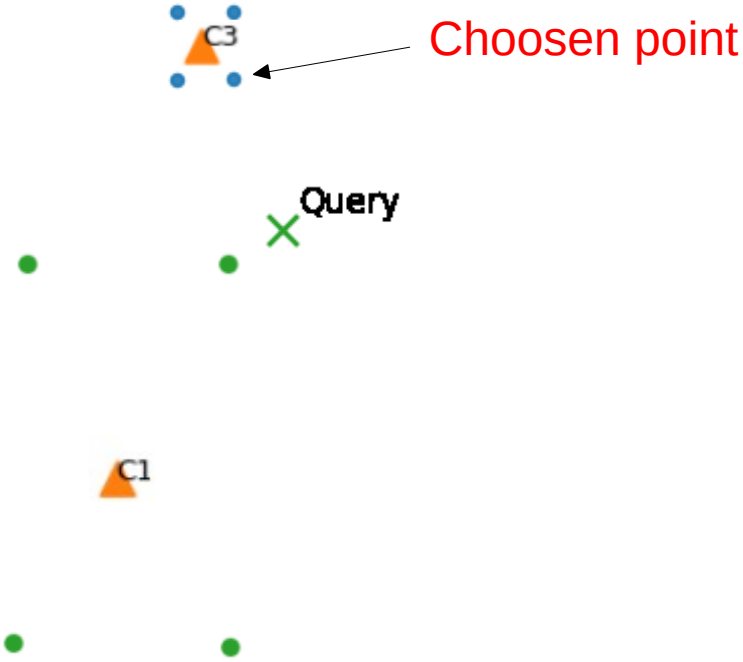
IVF: Query time

Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

Query is closest to C3, so lower right blue point is marked as closest



IVF: Query time

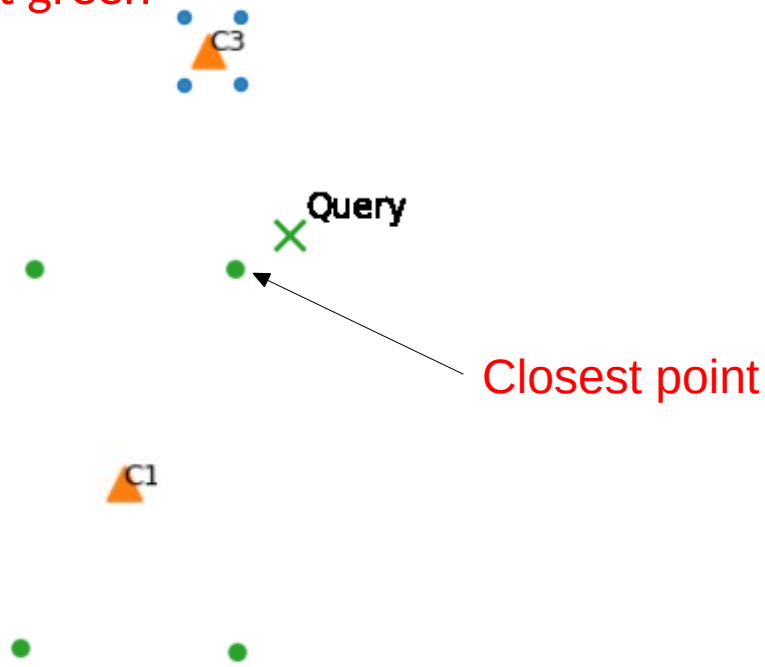
Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

Query is closest to C3, so lower right blue point is marked as closest

Correct is upper right green



IVF: Query time

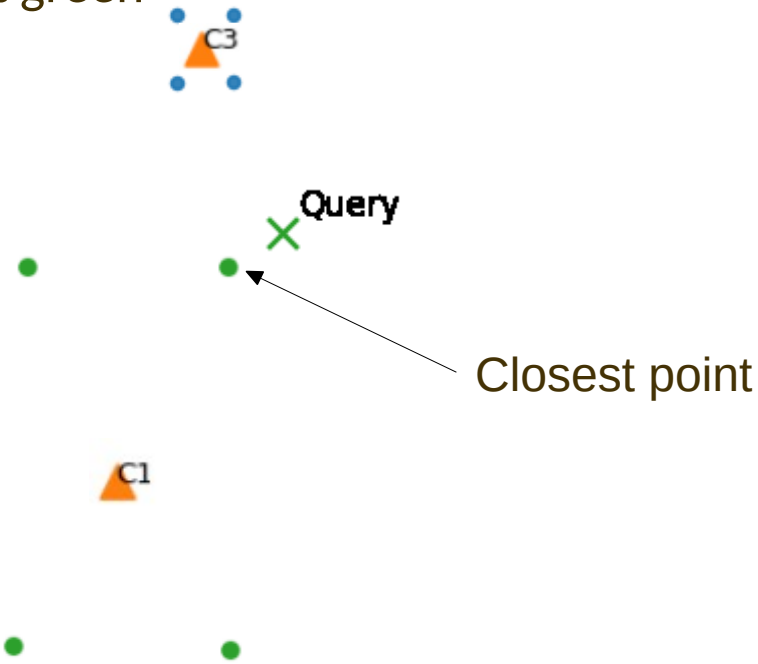
Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

Query is closest to C3, so lower right blue point is marked as closest

Correct is upper right green

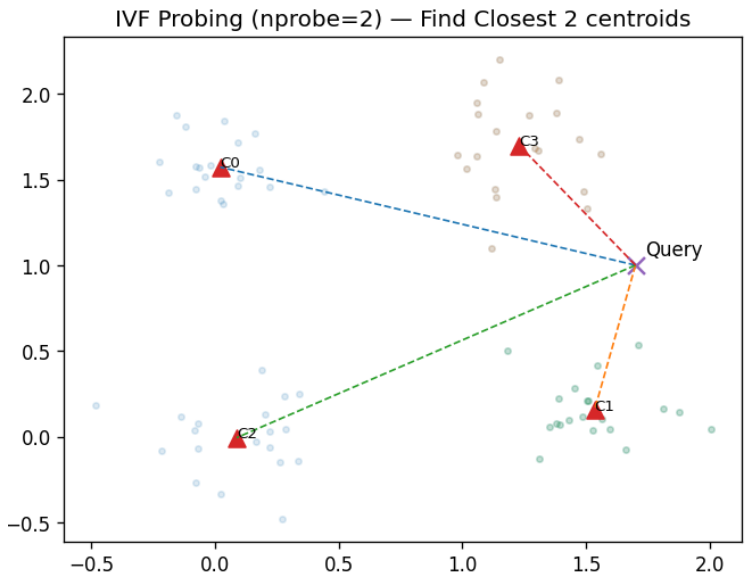


Partial solution is to allow searching more than 1 cluster

IVF: Query time

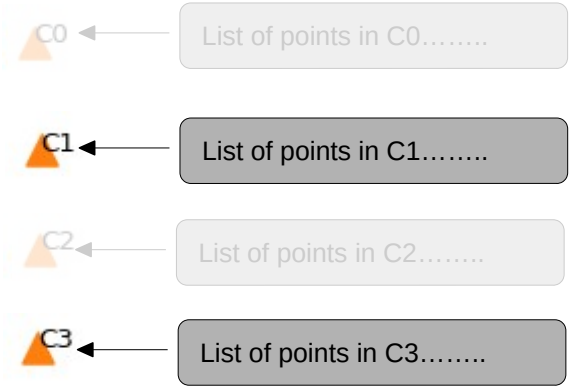
Ex. Find closest 3 points to Query

Find Closest **2** clusters



C1 and C3 are closest centroids

Use the Lists for C1 and C3 to get all the points to compare to

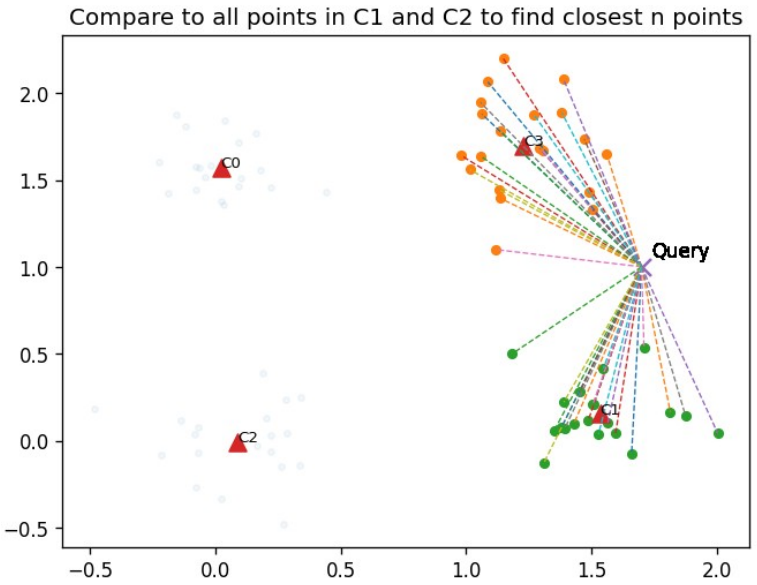


IVF: Query time

Ex. Find closest 3 points to Query

Find Closest 2 clusters

Calculate distances to all points in each cluster



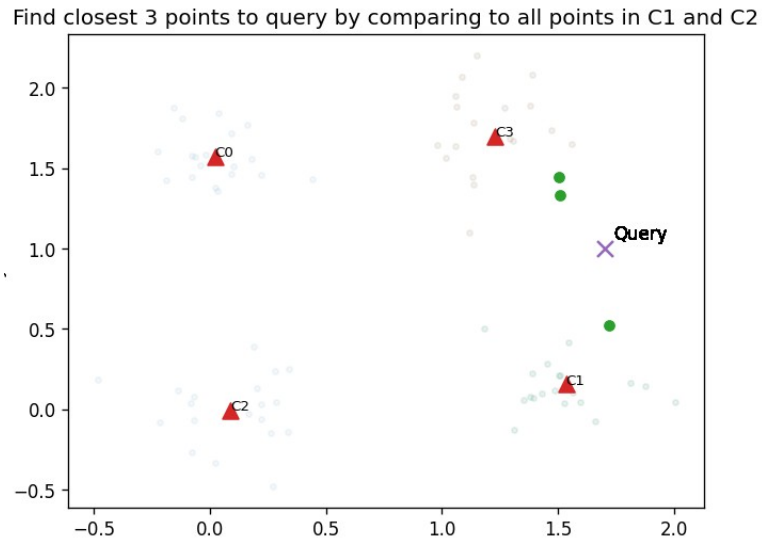
IVF: Query time

Ex. Find closest 3 points to Query

Find Closest 2 clusters

Calculate distances to all points in each cluster

Sort distances,Select closest 3 points



NSW: Navigable Small Worlds — Idea

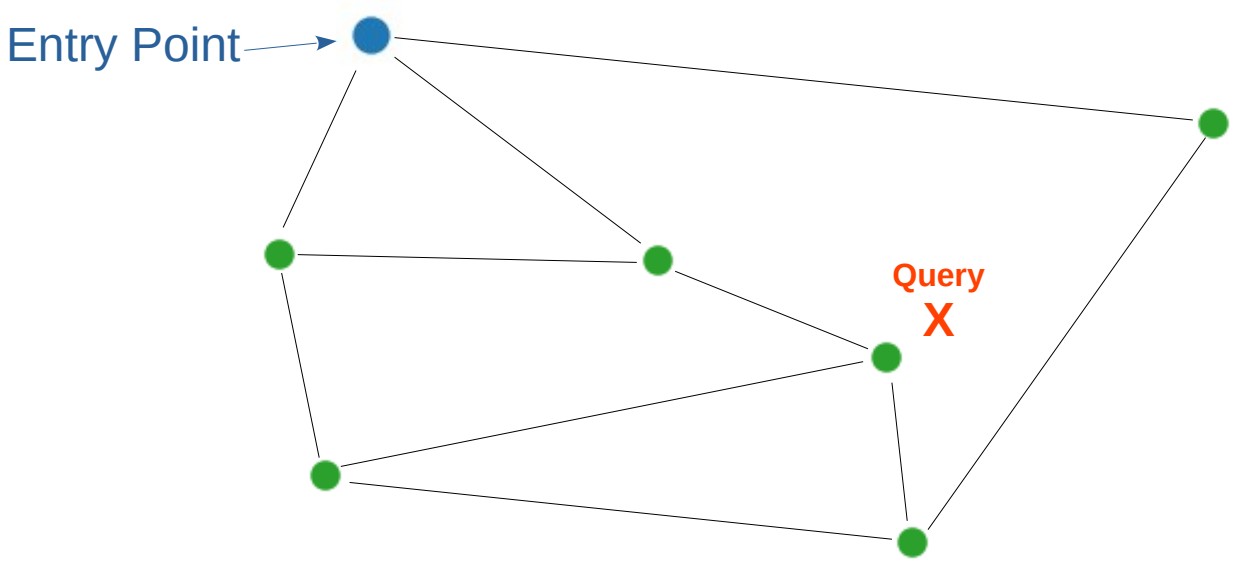
- Create a connected graph of all points
- Connections are made from each point to other points via ‘nearest neighbor’ algorithm. Connected points are called ‘friends’
- Limit the number of friends a node can have
- Each point keeps a list of its friends

NSW: Navigable Small Worlds — Search

1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2

NSW: Navigable Small Worlds — Search

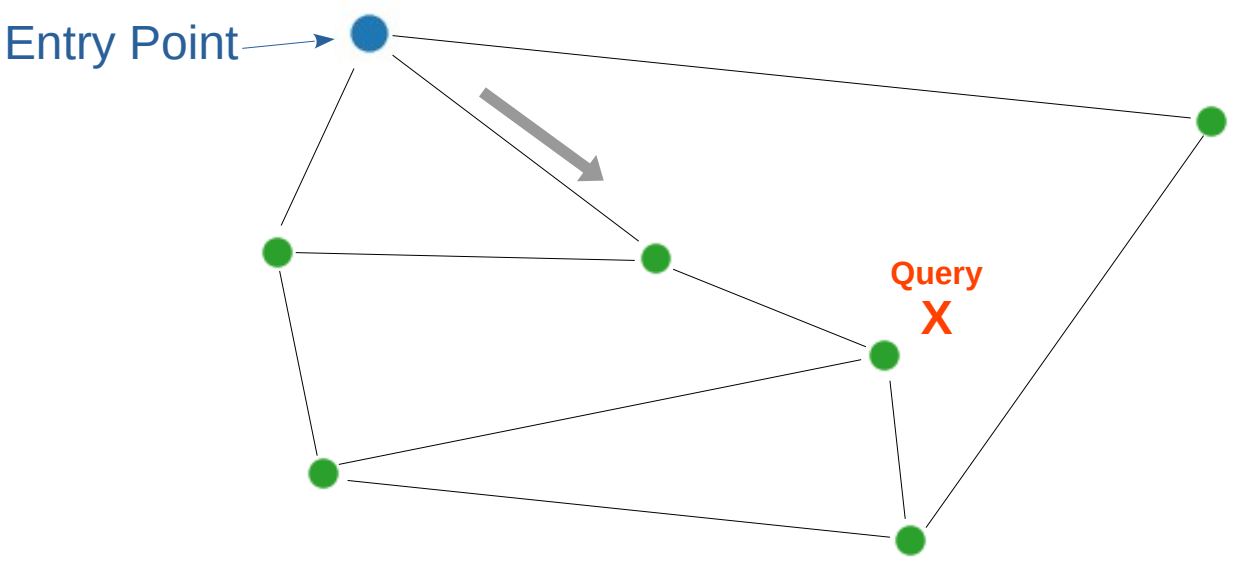
1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2



See https://www.pinecone.io/learn/series/faiss/hnsw/ for original image

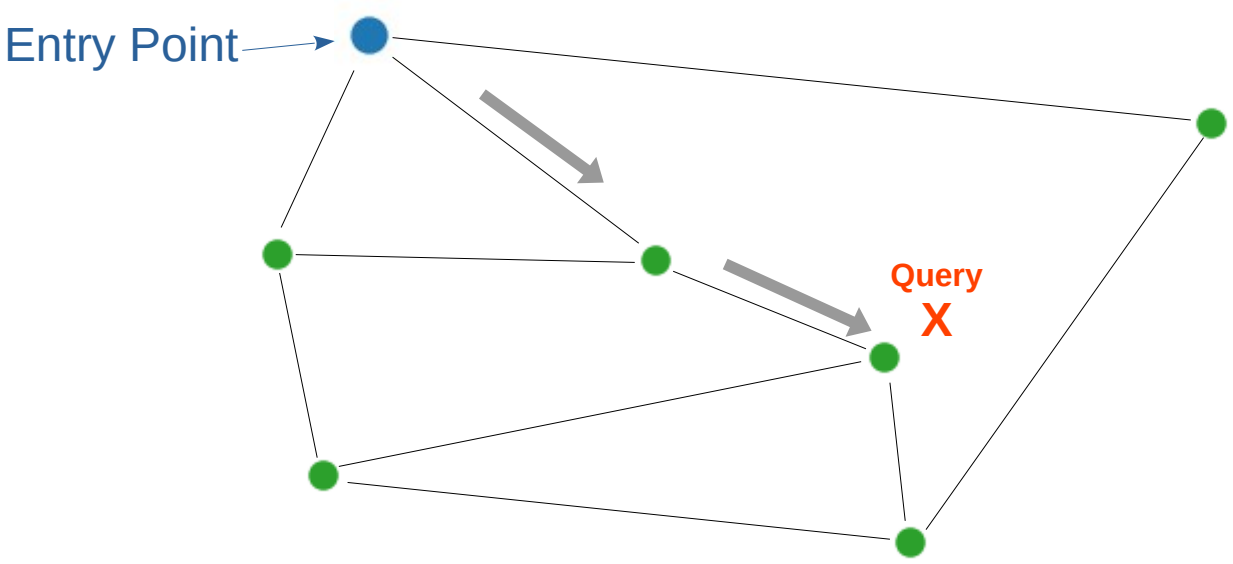
NSW: Navigable Small Worlds — Search

1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2



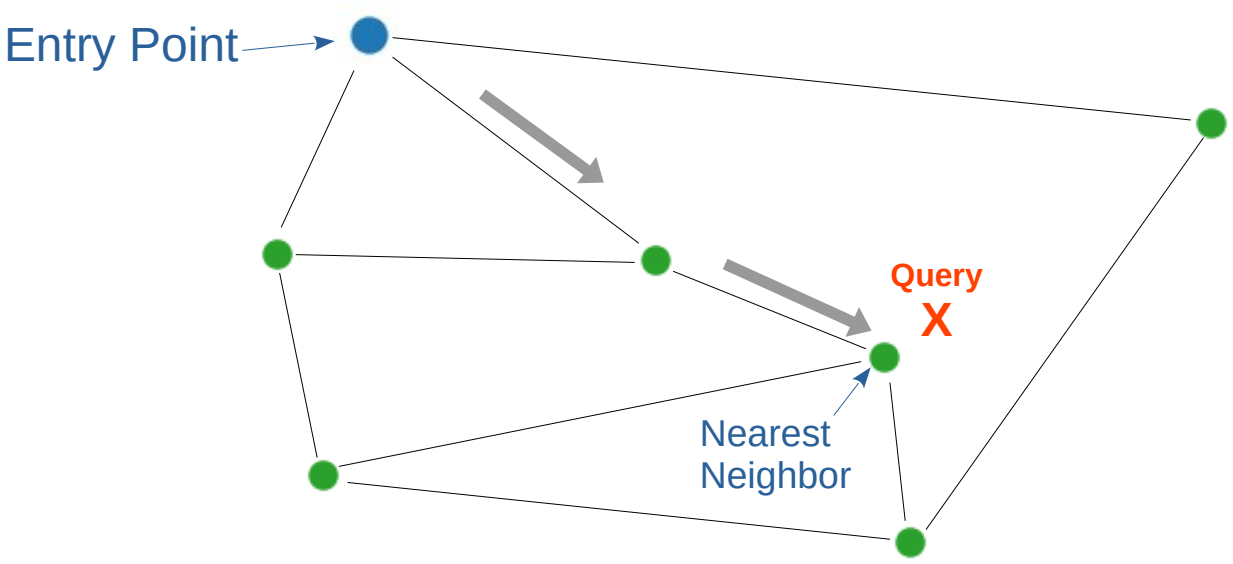
NSW: Navigable Small Worlds — Search

1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2



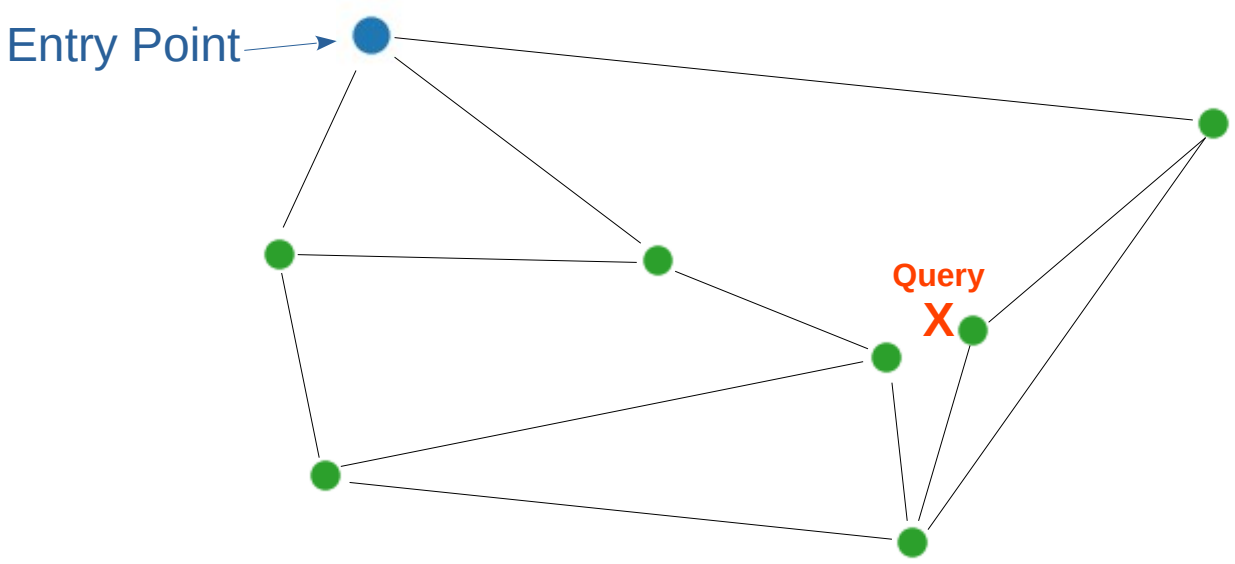
NSW: Navigable Small Worlds — Search

1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2



NSW: Navigable Small Worlds — Problem

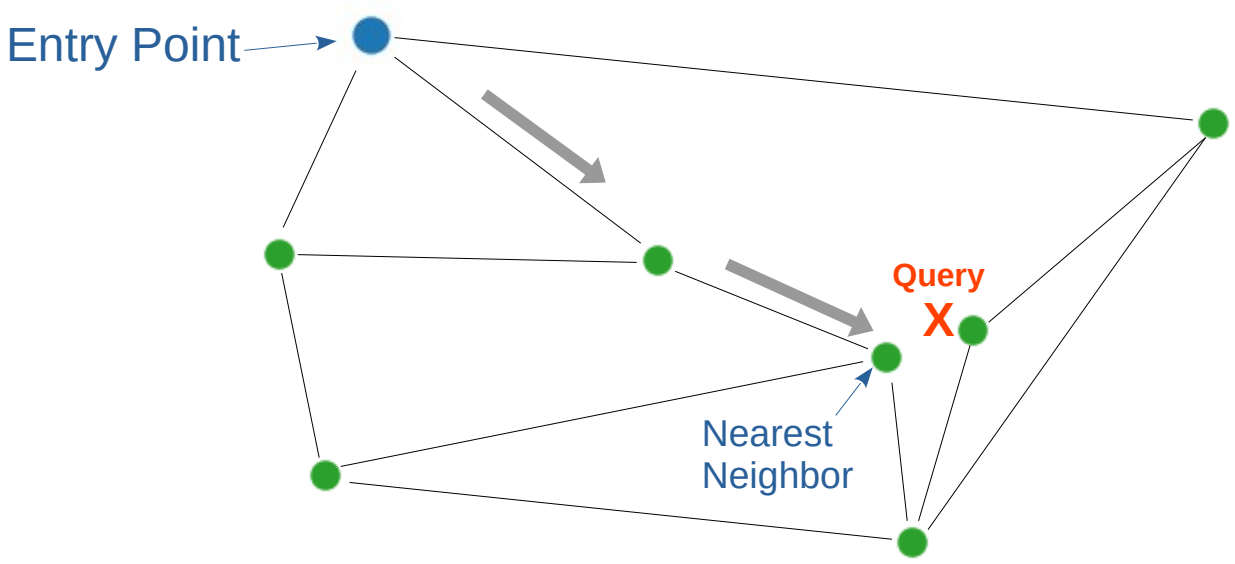
1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2



But NSW does not guarantee correct nearest neighbor, add 1 more node

NSW: Navigable Small Worlds — Problem

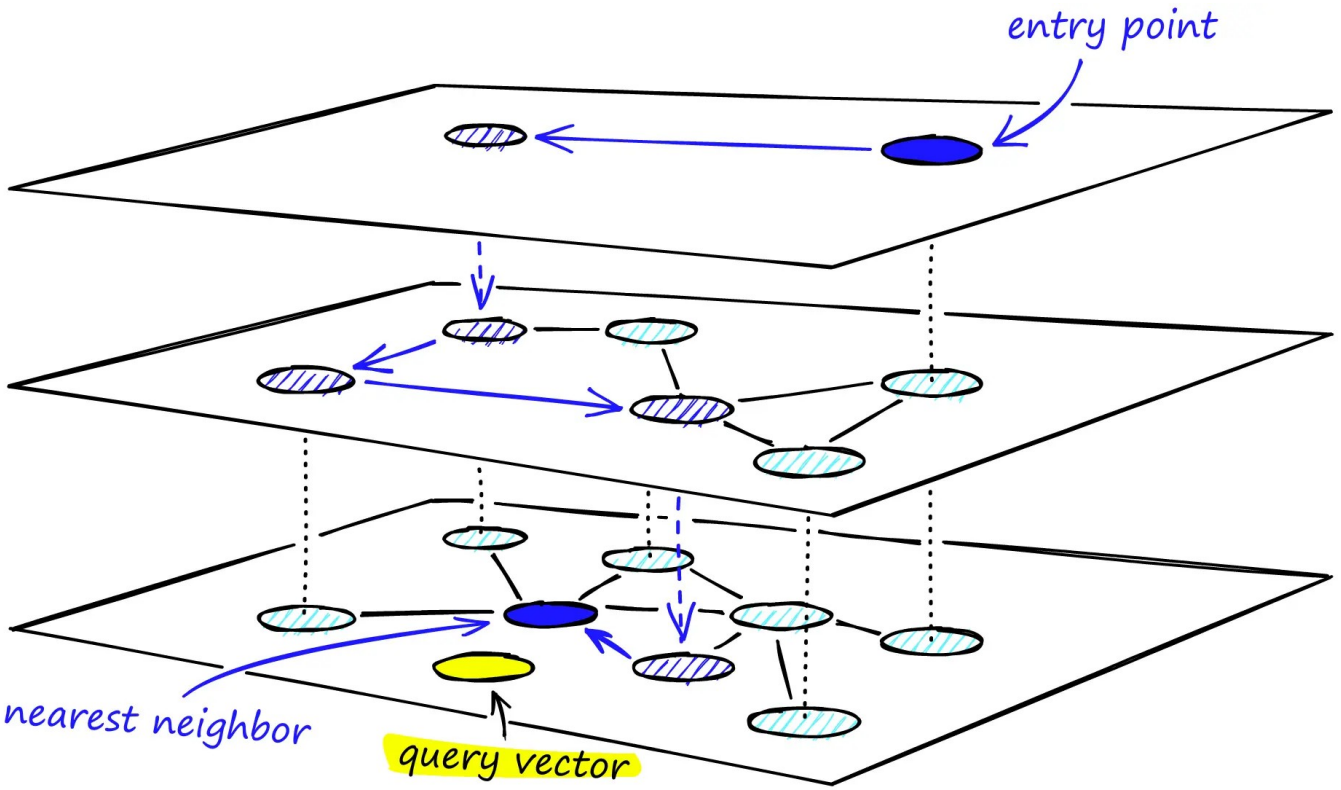
1. Start at pre-defined entry point
2. Greedy Search: choose path to friend that is closest to query
3. If no friends closer than current node we are done (a local minima)
4. Go to 2



But NSW does not guarantee correct nearest neighbor, add 1 more node

Algorithm never selects best node, NSW is an approximation

HNSW: Heiarchial Navigable Small Worlds



- Multi-layer graph; greedy search from top layer, descending through denser layers.
- Evolution of NSW

HNSW: Heiarchial Navigable Small Worlds-Build

SIMPLIFIED OVERVIEW

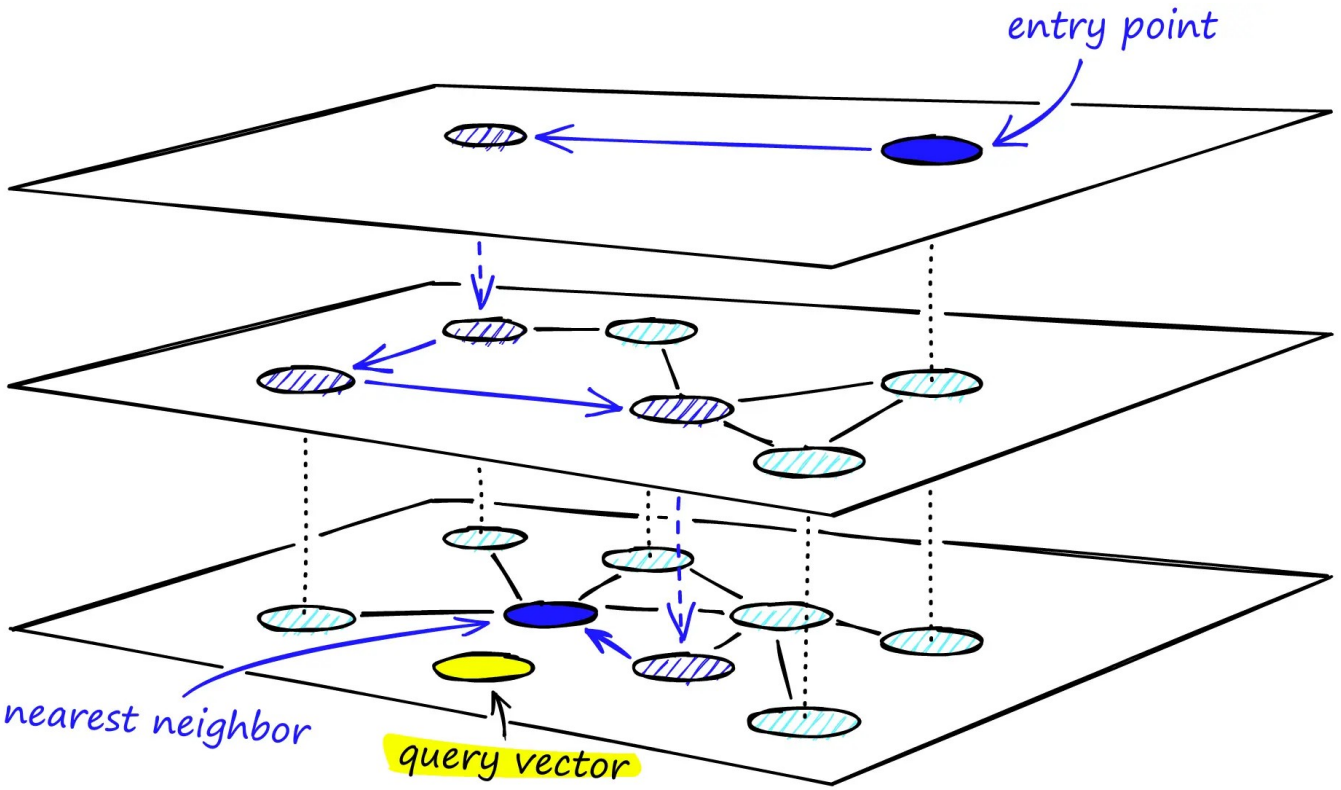
1. Start with single layer, a NSW graph with all points.
2. Generate a decreasing list of probabilities prob[numb_layers] (1 per layer)

   ex. prob=[.97, .025, .005]
3. Cur_layer =0
4. Nex_layer=Cur_layer+1
5. Use probabilistic insertion for all points in layer

   ex. pt1= np.random.rand() #for layer 1 if pt1<prob[1] pt1 inserted in layer 1
6. Get a smaller, distributed representation of lower layer in new layer
7. Build friend list for this layer
8. Stop when have enough layers, otherwise go to 4

Define entry point at top layer

HNSW: Heiarchial Navigable Small Worlds-Use



1. Start at entry
2. Traverse edges in each layer, greedily moving to the nearest vertex to Query (local minima)
3. Drop down a layer.
4. Go to step 2 unless you are on bottom layer (layer 0)

See https://www.pinecone.io/learn/series/faiss/hnsw/ for original image

Quick Comparison

Aspect	kNN (Exact)	IVF	HNSW
Query time	Highest (O(n))	Low, tunable (nprobe - #clusters)	Very low, tunable (efSearch*)
Build time	None	Moderate (k-means)	High (graph building)
Memory	Data only	Data + centroids	Data + graph links (higher)

- kNN will find exact match
- IVF and HNSW may not

*Efsearch controls greedyness of search, higher number means search more candidates per node

Summary

- kNN is exact but slow for large n
- IVF: partition + probe
- NSW: connected graph with greedy search
- HNSW: layered graph, fast greedy search